School of Future Tech

Case Study Report

on

Weather Data Aggregator

by

Jui Sudhir Tawde

150096725149

Index

1. Introduction to the Case Study

Weather information plays an important role in everyday life, influencing decisions related to travel, outdoor activities, agriculture, and logistics. With the availability of open weather APIs, real-time environmental data can be accessed and analyzed through web-based systems.

This case study focuses on the development of a **Weather Data Aggregator**, a web application that collects real-time temperature data for multiple cities and performs statistical analysis on the collected data. The system fetches temperature data for five major Indian cities: **Mumbai, Delhi, Kolkata, Bangalore, and Chennai**.

The application demonstrates how modern web technologies can interact with external APIs, process data in the browser using JavaScript, and visualize the results through charts and dynamic user interfaces. By combining asynchronous API fetching, data aggregation logic, and visualization techniques, the project simulates how real-time analytics dashboards operate in modern data-driven systems.

---

2. Problem Statement / Case Background (Abstract)

Background

Most users check weather data for individual cities separately using different websites or applications. Comparing temperatures across multiple cities requires manually switching between different platforms and gathering the data independently. This process is inefficient and time consuming.

Therefore, there is a need for a centralized dashboard that can **fetch weather information from multiple locations simultaneously and perform automatic analysis on the collected data**.

Abstract

This project presents the design and implementation of a **Multi-City Weather Data Aggregator** developed using JavaScript and modern frontend technologies.

The system retrieves real-time weather information using the **Open-Meteo API**, processes the temperature values, and calculates statistical measures such as:

- Average temperature
- Highest temperature

- Lowest temperature

The results are dynamically displayed in a dashboard interface containing city cards, statistical cards, an interactive map, and a bar chart visualization built using **Chart.js**.

The application also includes several user experience features such as:

- Dark / Light mode toggle
- Animated gradient background
- Cloud animation system
- Loader animation during data fetching
- Interactive hover effects
- Video carousel with weather-related content

This project demonstrates the integration of **API communication, asynchronous JavaScript programming, real-time data aggregation, and dynamic visualization in a browser-based environment.**

---

3. Case Study Design

The Weather Data Aggregator system is designed as a **frontend dashboard application** with multiple interconnected modules.

1. Data Acquisition

The system collects weather data from the **Open-Meteo API**, a free public weather API service.

Key features of data acquisition include:

- Using latitude and longitude coordinates for each city
- Fetching real-time temperature data
- Making multiple API calls simultaneously using **Promise.all()**
- Ensuring faster data retrieval through parallel execution

Cities used in the project:

- Mumbai
- Delhi
- Kolkata
- Bangalore
- Chennai

Each city request retrieves the **current weather temperature in Celsius**.

---

2. Data Processing and Aggregation

After the weather data is fetched from the API, the system extracts the temperature values and stores them in an array.

Example:

const temps = [28, 30, 26, 32, 29];

The system then performs statistical calculations:

Average Temperature

The average temperature is calculated using the formula:

Average = Sum of temperatures / Number of cities

The **reduce() function** is used to compute the sum.

Highest Temperature

The highest temperature among all cities is calculated using:

Math.max(...temps)

Lowest Temperature

The lowest temperature among all cities is calculated using:

Math.min(...temps)

These values are then displayed in the dashboard.

3. User Interface Display

The user interface is designed as a **dashboard layout** containing several components:

- City cards showing individual city temperatures
- Statistical cards showing:
    - Average temperature
    - Highest temperature
    - Lowest temperature
- Interactive map displaying city locations and temperature markers
- A chart displaying temperature comparison across cities

The UI updates dynamically after the user clicks the **Fetch Weather Data** button.

4. Data Visualization

Temperature data is visualized using **Chart.js**, a JavaScript charting library.

The visualization includes:

- A **bar chart** showing temperature values for each city
- Dynamic axis labels and colors
- Automatic chart updates when new data is fetched
- Chart theme adaptation for light and dark mode

This helps users easily compare weather conditions across cities.

5. User Experience Enhancements

Several UI and animation features were added to improve the visual appeal and interactivity of the application.

These include:

- Animated gradient sky background

- Floating cloud animation system
- Glassmorphism styled cards
- Button hover and click animations
- Loader animation during API fetching
- Dark / Light mode theme toggle
- Video carousel for weather-related educational content

These enhancements make the dashboard more engaging and interactive.

---

4. Methods & Technologies Applied in the Case Study

Key Methods Used

1. Asynchronous JavaScript

The system uses asynchronous programming to retrieve data from external APIs.

Key techniques used:

- `fetch()` API
- `async/await` syntax
- `Promise.all()` for parallel API calls

Parallel fetching significantly improves performance.

---

2. Data Aggregation Techniques

Several JavaScript methods are used to process and analyze temperature data:

- `map()` to extract temperature values
- `reduce()` to compute the sum

- `Math.max()` to find the highest temperature
- `Math.min()` to find the lowest temperature

These techniques allow efficient statistical analysis of the dataset.

---

3. Dynamic DOM Manipulation

The application dynamically updates the webpage content without reloading the page.

Methods used include:

- `querySelector()`
- `querySelectorAll()`
- `textContent` updates
- Event listeners for button clicks

This enables real-time updates to the dashboard.

---

4. Data Visualization

Temperature data is represented visually using **Chart.js**.

Features include:

- Bar chart rendering
- Dynamic axis color adjustments
- Responsive chart resizing
- Theme-based chart styling

Technology Stack Used

Frontend Technologies

- **HTML5** – Structure of the application

- **CSS3** – Styling and animations
- **JavaScript (ES6)** – Logic and data processing

Libraries and Tools

- **Chart.js** – Data visualization
- **Leaflet.js** – Interactive map display

API Used

- **Open-Meteo Weather API**

Design Techniques

- Glassmorphism UI
- Animated gradients
- Responsive grid layout
- Theme switching with CSS variables

Development Tools

- Visual Studio Code
- Live Server extension

---

5. Case Study Implementation Details and Snapshots

Implementation Structure

The project is organized into three main files:

1. index.html

This file contains:

- Dashboard layout structure
- City temperature cards
- Statistics cards
- Map container
- Chart canvas
- Fetch button
- Loader animation
- Video carousel section

---

2. style.css

This file handles all visual styling including:

- Gradient background animation
- Cloud animation system
- Card layouts and glass effects
- Button hover animations
- Dark / Light mode styles
- Loader animation

---

3. script.js

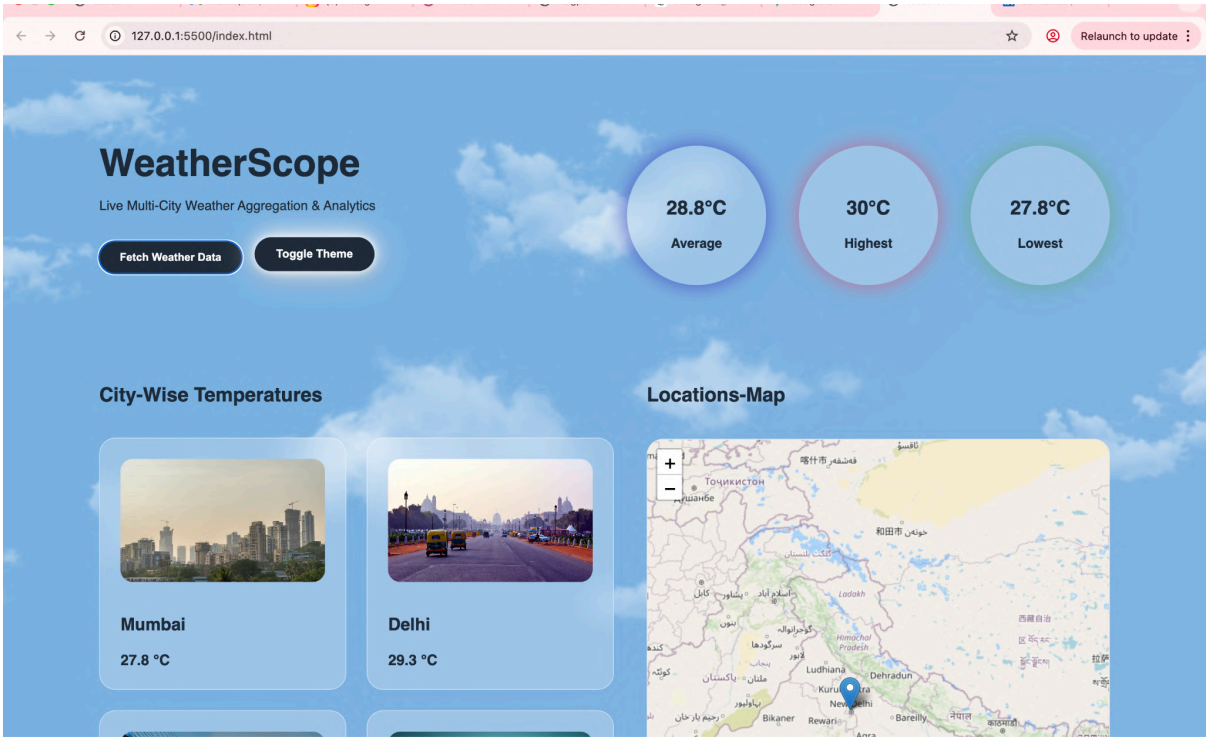
This file contains the application logic:

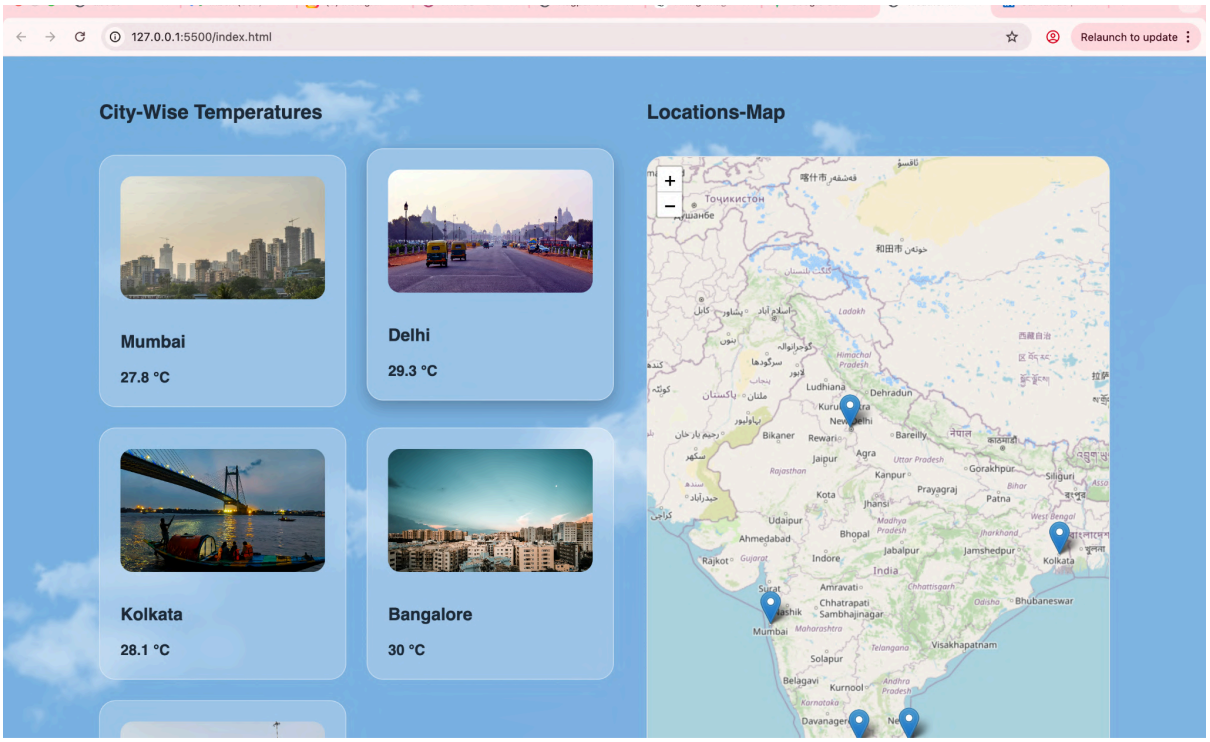
- City coordinate dataset
- Weather API fetch logic
- Promise.all() implementation

- Temperature extraction
- Aggregation calculations
- Chart rendering
- Theme toggle logic
- Map marker updates
- Video carousel functionality
- Loader display logic

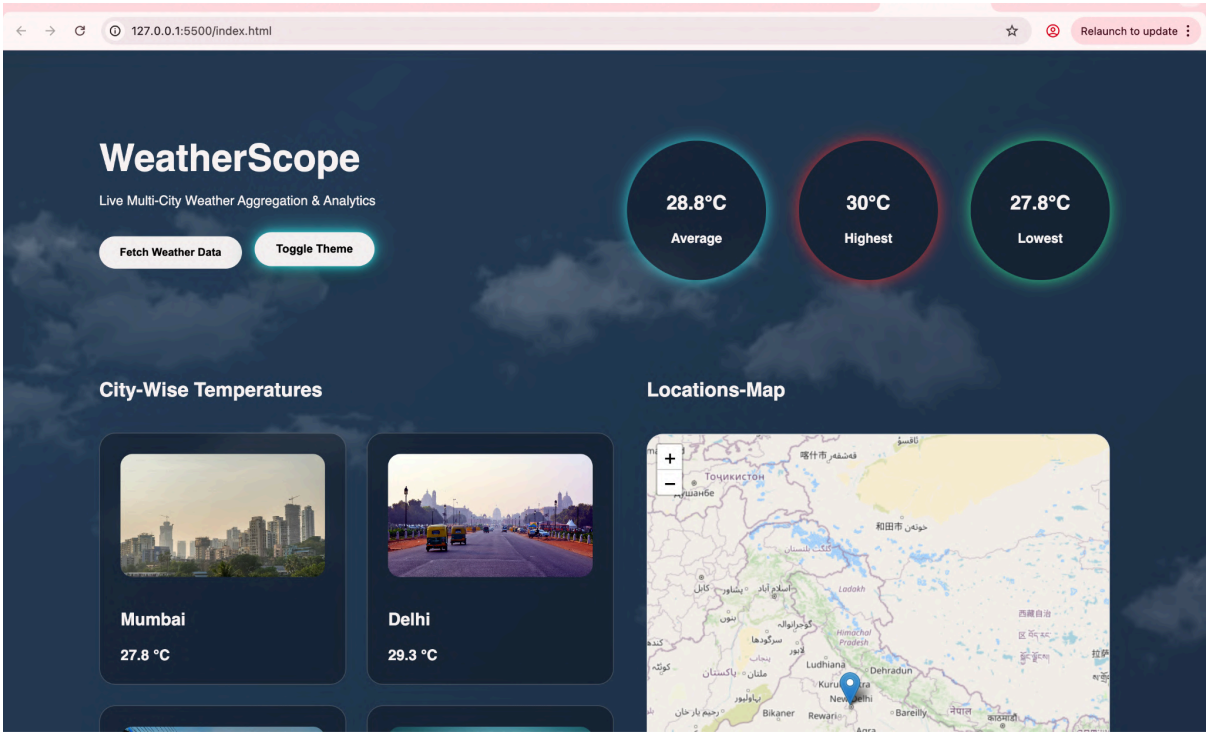
Snapshots to Include

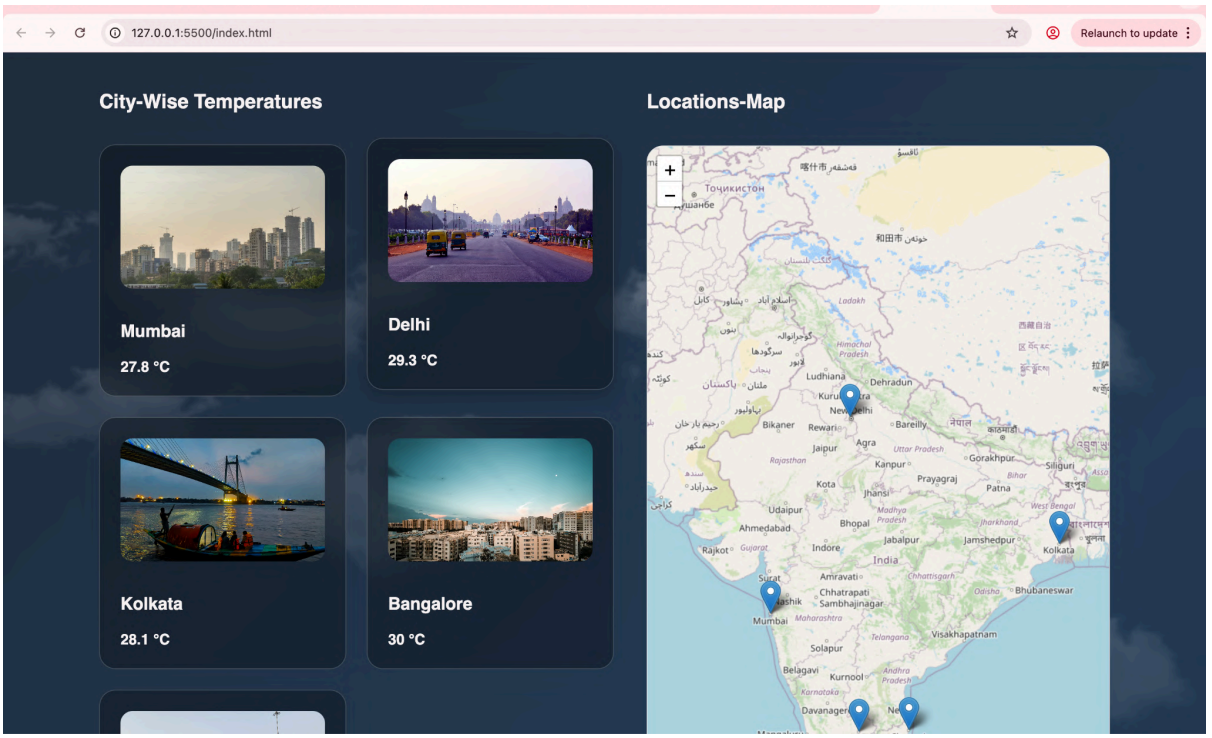
- Dashboard (Light Mode)





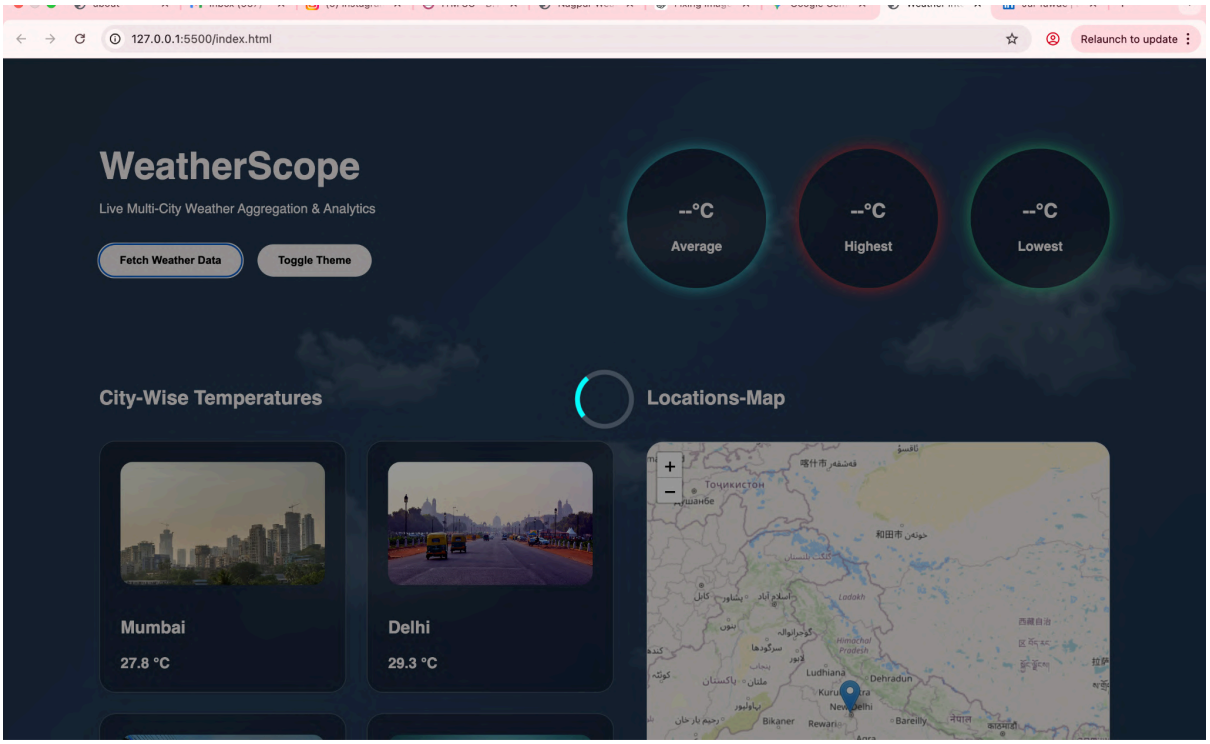
- Dashboard (Dark Mode)



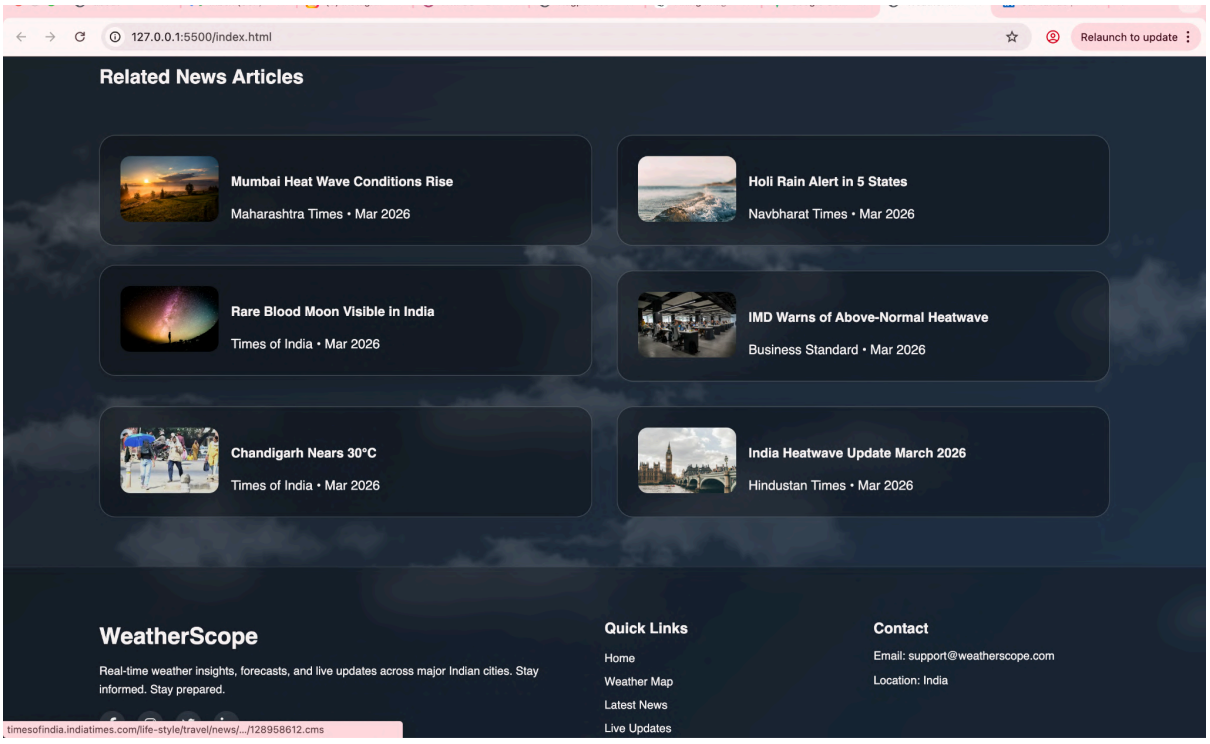


- Chart visualization

- Loader animation



- News Articles Section



- Code snippet of `Promise.all()` implementation

```
// Create API requests for all cities
const requests = cities.map(city =>
  fetch(`https://api.open-meteo.com/v1/forecast?latitude=${city.lat}&longitude=${city.lon}&current_weather=true`)
    .then(res => res.json())
);

// PARALLEL FETCH (important requirement)
const results = await Promise.all(requests);
```

---

6. Case Study Results and Conclusion

Findings

The project successfully demonstrates the use of modern web development techniques to build a real-time analytics dashboard.

Key observations include:

- **Parallel API fetching** using Promise.all() significantly reduces waiting time.
- Real-time aggregation provides quick comparative analysis of city temperatures.
- Interactive charts improve understanding of temperature differences.
- UI animations and themes enhance the overall user experience.
- Modular JavaScript code structure improves maintainability.

Conclusion

This case study demonstrates the development of a **Weather Data Aggregator Dashboard** that combines real-time API data retrieval, asynchronous JavaScript programming, statistical data aggregation, and visual data representation.

The system successfully performs:

- Multi-city weather data fetching
- Temperature comparison across cities
- Statistical calculations (average, highest, lowest)
- Dynamic chart visualization

- Interactive dashboard presentation

The project showcases how frontend technologies can be used to build **real-time data-driven applications** similar to professional analytics dashboards.

Future improvements could include:

- Humidity and wind speed tracking
- Weather condition icons
- Forecast trend graphs
- Historical weather storage
- Backend integration for data persistence.

---

7. References

- Open-Meteo API Documentation
  https://open-meteo.com/en/docs
- Chart.js Official Documentation
  https://www.chartjs.org/docs
- MDN Web Docs – Fetch API
  https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Web Docs – CSS Animations
  https://developer.mozilla.org/en-US/docs/Web/CSS/animation
